# Azure Data Engineering Project Flow

*End-to-End Implementation with Medallion Architecture & Azure Data Factory*

## 1. Azure Data Engineering Foundation & Architecture

### 1.1 Azure Data Engineering Overview

Azure Data Engineering is a cloud-based approach to designing, building, and maintaining data infrastructure on Microsoft Azure. It encompasses the entire data lifecycle from ingestion through processing to analytics and visualization. Azure provides a comprehensive suite of tools and services that enable organizations to build scalable, secure, and cost-effective data solutions.

Key Azure data engineering services include:

- Azure Data Lake Storage (ADLS Gen2) - Scalable data lake infrastructure
- Azure Data Factory (ADF) - Serverless data orchestration and ETL
- Azure Synapse Analytics - Integrated analytics platform
- Azure Databricks - Apache Spark-based big data processing
- Azure Stream Analytics - Real-time data processing
- Azure Event Hubs - Distributed event streaming platform

### 1.2 Medallion Architecture - Bronze, Silver, Gold Layers

The Medallion Architecture is a best-practice data organization pattern that structures data in three distinct layers within Azure Data Lake Storage, each serving a specific purpose in the data pipeline.

#### 1.2.1 Bronze Layer (Raw Data)

The Bronze layer serves as the initial landing zone for all raw, unprocessed data ingested from source systems. This layer maintains data fidelity and historical records.

- Purpose: Store raw data exactly as received from source systems
- Data Format: Parquet, CSV, JSON, or native format from source
- Processing: Minimal to no transformation; focus on landing data efficiently
- Retention: Usually maintains full historical data
- Examples: Raw sales transactions, API responses, log files, database exports
- Azure Service: Azure Data Lake Storage (ADLS Gen2)

Bronze layer best practices in Azure:

- Use ADLS Gen2 hierarchical namespace for optimal performance
- Implement folder structure: /bronze/{source_system}/{data_domain}/{table_name}/YYYY/MM/DD/
- Use Parquet format for efficient storage and query performance
- Implement time-based partitioning for easy data retention management
- Set appropriate access controls using Azure AD and RBAC

### 1.2.2 Silver Layer (Cleaned & Standardized Data)

The Silver layer contains cleaned, validated, and standardized data. Data is transformed to meet quality standards and conforms to organizational data models and naming conventions.

- Purpose: Provide a single source of truth for cleaned data
- Data Quality: Deduplicated, validated, null values handled
- Transformations: Data type conversions, standardization, enrichment
- Performance: Optimized for analytics and business logic
- Schema: Defined schemas using Delta format
- Azure Service: Azure Synapse Analytics, Databricks, or Azure Data Factory

Silver layer transformation activities:

- Remove duplicates using hash functions on key columns
- Validate data quality against predefined rules (null checks, range checks)
- Standardize formats (dates, phone numbers, addresses)
- Implement surrogate keys for dimensions
- Join related entities and denormalize for analytics
- Implement SCD Type 1 and Type 2 for slowly changing dimensions

### 1.2.3 Gold Layer (Business-Ready Analytics)

The Gold layer contains optimized, aggregated data ready for direct consumption by analytics, reporting, and machine learning applications. Data is organized in a business-friendly dimensional model.

- Purpose: Provide business metrics and analytics-ready datasets
- Format: Star schema with fact and dimension tables
- Optimization: Pre-aggregated, indexed, and denormalized for query performance
- Consumers: Power BI, Tableau, machine learning models, business applications
- Latency: Typically updated on schedule (hourly, daily)
- Azure Service: Azure Synapse Analytics SQL Pools or Azure SQL Database

Gold layer data structures:

- Fact tables containing metrics, dimensions, and measure values
- Dimension tables with business context and attributes
- Aggregate tables for common business queries
- Time-based snapshots for temporal analysis

# 2. Azure Data Lake Storage (ADLS Gen2) Setup & Configuration

## 2.1 ADLS Gen2 Foundation

Azure Data Lake Storage Gen2 is a set of capabilities dedicated to big data analytics, built on Azure Blob Storage. It provides hierarchical namespace, security features, and Azure ecosystem integration.

### 2.1.1 Creating and Configuring Storage Accounts

- Create Azure Storage Account with ADLS Gen2 capabilities enabled
- Enable Hierarchical Namespace (required for ADLS Gen2)
- Configure performance tier: Standard or Premium
- Set replication strategy: LRS (Local Redundancy), GRS (Geo-Redundancy)
- Enable soft delete for data recovery capability

- Configure blob access tier: Hot, Cool, or Archive

### 2.1.2 Container and Folder Structure Design

- Create containers for each environment: dev, staging, production
- Implement medallion layer structure: /bronze, /silver, /gold
- Use data domain organization: /bronze/sales, /bronze/marketing
- Implement date-based partitioning: /YYYY/MM/DD/HH/
- Maintain consistent naming conventions across all layers

### 2.1.3 Access Control and Security

- Configure managed identities for Azure services
- Implement Azure AD integration for user access
- Use RBAC roles: Storage Blob Data Owner, Reader, Contributor
- Enable Azure Private Endpoints for secure network access
- Configure Network Security Groups (NSG) rules
- Implement encryption at rest using customer-managed keys (optional)
- Enable encryption in transit using TLS 1.2 or higher

## 2.2 ADLS Gen2 Performance Optimization

### 2.2.1 Partitioning Strategy

- Implement date-based partitioning for time-series data
- Use hash-based partitioning for even data distribution
- Partition by business entities (customer_id, product_id)
- Aim for 128 MB to 256 MB file sizes for optimal performance

### 2.2.2 File Format Selection

- Parquet: Optimal for analytics with columnar storage, compression
- Delta format: Built-in ACID transactions, schema enforcement
- CSV: Simple but requires more storage space and slower queries
- JSON: Semi-structured data, preserves schema flexibility

### 2.2.3 Data Lifecycle Management

- Implement tiered storage: Hot (0-30 days), Cool (30-90 days), Archive (90+ days)
- Use lifecycle management policies to automatically move data between tiers
- Delete data according to retention policies and compliance requirements
- Implement backup and disaster recovery strategies

# 3. Azure Data Factory (ADF) - Data Orchestration & ETL

## 3.1 Azure Data Factory Architecture

Azure Data Factory is a fully managed, serverless data integration service that enables you to create complex ETL and data orchestration workflows. ADF provides a visual interface and code-based options for building data pipelines.

### 3.1.1 ADF Core Concepts

- Pipelines: Container for activities that perform data operations

- Activities: Individual tasks like Copy Data, Data Flow, Spark Job
- Datasets: Named references to data structures in linked services
- Linked Services: Connection information to external data stores
- Integration Runtime: Compute infrastructure for executing activities
- Triggers: Scheduling mechanisms for pipeline execution

### 3.1.2 Integration Runtime Configuration

Integration Runtimes are the compute infrastructure used by ADF to execute activities and transformations.

- Azure Integration Runtime: For cloud-to-cloud scenarios, auto-resolving
- Self-Hosted Integration Runtime: For on-premises data sources and networks
- Azure SSIS Integration Runtime: For legacy SSIS package execution
- Managed Virtual Network: Private connectivity with private endpoints

## 3.2 ADF Pipeline Design for Medallion Architecture

### 3.2.1 Bronze Layer Ingestion Pipeline

The Bronze ingestion pipeline in ADF handles the initial data landing without transformation. It copies data from source systems to the Bronze layer maintaining data fidelity.

- Source: Database (Azure SQL, On-premises SQL Server), APIs, File shares
- Activity: Copy Data activity with source and sink datasets
- Sink Configuration: ADLS Gen2 Bronze containers with date-based paths
- Error Handling: Implement retry logic and error notification
- Validation: Row count and checksum validation post-copy
- Scheduling: Triggered by schedule or event-based mechanism

### 3.2.2 Silver Layer Transformation Pipeline

Silver layer pipelines in ADF perform data cleansing, validation, and standardization. These pipelines can use Data Flows, Databricks, Spark pools, or SQL scripts.

- Data Flow Activity: Visual transformation interface for large-scale data
- Databricks Activity: Execute Spark notebooks for complex transformations
- SQL Script Activity: Run SQL stored procedures in Synapse pools
- Transformations: Deduplication, null handling, type conversion, enrichment
- Output: Silver layer Parquet or Delta files
- Validation: Data quality checks, schema validation

### 3.2.3 Gold Layer Aggregation Pipeline

Gold layer pipelines create business-ready datasets with pre-aggregated metrics and dimensional models. These typically load into dedicated Synapse SQL pools.

- Source: Silver layer cleaned data
- Transformations: Aggregations, calculations, dimensional joins
- Activity: Polybase loading or Bulk Insert for efficient loading
- Performance: Implement incremental loads using watermark patterns
- Sink: Dedicated SQL pools or Azure SQL Database

## 3.3 ADF Data Flow for ETL Transformations

### 3.3.1 Mapping Data Flows

- Visual transformation interface without code writing
- Source transformation: Load data from Bronze layer
- Filter transformation: Remove unwanted rows
- Select transformation: Choose and rename columns
- Aggregate transformation: Group by and calculate metrics
- Join transformation: Combine multiple data sources
- Pivot/Unpivot transformations: Reshape data structure
- Sink transformation: Write to Silver or Gold layers

### 3.3.2 Data Flow Performance Optimization

- Use staging transformation for intermediate aggregations
- Implement column projection to reduce data volume
- Use filter transformation early to reduce downstream processing
- Optimize cluster size for large transformations
- Monitor execution time and adjust cluster configuration

## 3.4 ADF Triggering and Scheduling

### 3.4.1 Schedule-Based Triggers

- Tumbling Window Trigger: Fixed interval execution (hourly, daily)
- Scheduling: Define start time, end time, frequency, and interval
- Backfill: Execute for historical periods automatically
- Examples: Daily full load at 2 AM, hourly incremental loads

### 3.4.2 Event-Based Triggers

- Event Grid Trigger: Respond to storage account events (file creation)
- Custom Event Triggers: Trigger on application events
- Chained Pipelines: Execute one pipeline upon another's completion
- Dependencies: Set up pipeline dependencies for sequencing

## 3.5 ADF Monitoring and Error Handling

### 3.5.1 Pipeline Monitoring

- Monitor panel in ADF UI shows real-time execution status
- Track activity-level details: duration, rows processed, errors
- Enable detailed logging to Azure Monitor and Log Analytics
- Set up alerts for pipeline failures or long execution times

### 3.5.2 Error Handling and Recovery

- Configure retry policies: Attempt, BackoffInterval, MaxRetries
- Implement error notification via Logic Apps or Functions
- On Failure hooks: Execute cleanup or notification activities
- Implement idempotent transformations for safe retries
- Use checkpoints in long-running pipelines

# 4. Data Ingestion - Source to Bronze Layer

## 4.1 Batch Data Ingestion with ADF

### 4.1.1 Database Source Ingestion

Ingesting data from relational databases is a common scenario in data engineering. ADF provides native connectors for various database platforms.

- Azure SQL Database: Use SQL Database linked service with connection string
- SQL Server: On-premises via Self-Hosted Integration Runtime
- PostgreSQL/MySQL: Native connectors available
- Oracle: Full JDBC-based connector support
- Copy activity configuration with pagination for large tables

### 4.1.2 File-Based Source Ingestion

- SFTP servers: Secure file transfer protocol sources
- Azure Blob Storage: Intra-Azure data movement
- Azure SharePoint/OneDrive: Office 365 data integration
- FTP/HTTP servers: HTTP-based file download
- Implement wildcard patterns for file discovery

### 4.1.3 API-Based Data Ingestion

Many modern applications expose data through REST or GraphQL APIs. ADF provides multiple mechanisms for API-based data ingestion.

- Web Linked Service: Generic HTTP endpoint support
- OAuth 2.0 authentication for secured APIs
- Azure Functions: Custom activities for complex API logic
- Pagination handling for large result sets
- Rate limiting management and retry logic

## 4.2 Real-Time Data Ingestion Patterns

### 4.2.1 Azure Event Hubs Integration

Event Hubs provide a highly scalable, distributed event streaming platform. Integrate with ADF for streaming scenarios.

- Consumer Groups: Multiple independent consumers from same stream
- Partitions: Distribute data for parallelism (typically 4-8 partitions)
- Stream Analytics: Real-time processing of incoming events
- Databricks Streaming: Structured Spark Streaming on event data
- ADLS integration: Stream events directly to Bronze layer

### 4.2.2 Azure IoT Hub Integration

- Device-to-cloud telemetry: Ingest IoT device data
- Built-in endpoint: Compatible with Event Hubs consumer API
- Device provisioning: Azure IoT Hub Device Provisioning Service
- Message enrichment: Add context before landing in Bronze layer

### 4.2.3 Change Data Capture (CDC)

CDC enables capturing only changed data since the last ingestion, improving efficiency for incremental loads.

- SQL Server CDC: Native change tracking in SQL Server
- Transactional Replication: Microsoft SQL Server built-in feature
- Logical Decoding: PostgreSQL change capture mechanism
- ADF Change Data Capture: Copy activity native CDC support
- Advantage: Reduced network traffic, faster incremental loads

## 4.3 Data Quality Checks at Ingestion

### 4.3.1 Schema Validation

Validate that incoming data matches expected schema before landing in Bronze layer.

- Column count validation
- Data type compatibility checks
- Required field presence validation
- Use Mapping Data Flow validation transformation

### 4.3.2 Completeness Checks

- Row count comparison: Source vs. ingested
- Null value detection in critical columns
- Date range validation for time-series data
- Implement lookup activities for referential integrity

### 4.3.3 Freshness Monitoring

- Track last modified timestamps from sources
- Alert on stale data (no updates for expected interval)
- Implement data arrival SLA monitoring
- Provide dashboard visibility into ingestion freshness

# 5. Silver Layer - Data Cleaning & Standardization

## 5.1 Data Quality Transformation Framework

### 5.1.1 Deduplication Logic

Remove duplicate records that may exist in source systems or result from multiple ingestion attempts.

- Row-level deduplication: Hash-based approach on all columns
- Key-based deduplication: Hash on business key columns
- SCD handling: Determine which duplicate is current version
- Implementation: Window functions with ROW_NUMBER() in SQL
- Alternative: Data Flow Group By transformation

### 5.1.2 Null Value Handling

- Identify null values in data quality profiling
- Determine treatment strategy: Remove, fill, or flag
- Domain knowledge: Use defaults or look-up tables

- Create quality flags for rows with imputed values
- Implement null handling transformations in Data Flow

### 5.1.3 Data Type Standardization

- Convert string dates to standard DATE or TIMESTAMP format
- Normalize phone numbers, postal codes, addresses
- Handle encoding issues in text fields
- Convert currency values to consistent decimal precision
- Implement type casting with error handling

## 5.2 Data Enrichment in Silver Layer

### 5.2.1 Dimension Enrichment

Add business context by joining with dimension tables.

- Customer dimension: Add customer details, segment, status
- Product dimension: Add product category, attributes
- Date dimension: Add fiscal periods, holiday flags
- Lookup table joins for standardized values

### 5.2.2 Calculated Fields

- Business metrics: Revenue, quantity, profit calculations
- Derived attributes: Age groups, tenure categories
- Temporal calculations: Days since last purchase, quarters
- Aggregated values: Customer lifetime value, purchase frequency

### 5.2.3 Flag and Quality Indicators

- Data quality flags: Complete, partially complete, incomplete
- Anomaly flags: Outliers detected based on statistical rules
- Freshness flags: Recently updated vs. stale data
- Record lineage: Source system, load timestamp, version

## 5.3 Slowly Changing Dimensions (SCD) Implementation

### 5.3.1 SCD Type 1 - Overwrite

Overwrite existing attributes with new values. No historical tracking.

- Use case: Status changes, location updates
- Implementation: Direct update or delete-insert
- Performance: Fastest, requires minimal storage

### 5.3.2 SCD Type 2 - Add New Row

Maintain full history by creating new rows for changes with effective dates.

- Use case: Price changes, organizational changes
- Columns: effective_date, end_date, is_current flags
- Implementation: Surrogate keys for dimension relationships
- Query challenge: Always filter for is_current = true

### 5.3.3 SCD Type 3 - Add Column

Keep limited history by adding columns for previous values.

- Use case: Status changes with prior value comparison
- Columns: current_value, previous_value
- Storage efficient but limited history depth

# 6. Gold Layer - Analytics-Ready Data Models

## 6.1 Dimensional Modeling for Analytics

### 6.1.1 Star Schema Design

Star schema organizes data in a denormalized structure with a central fact table surrounded by dimension tables.

- Fact Table: Quantitative measures, foreign keys to dimensions
- Dimension Tables: Descriptive attributes, slowly changing dimensions
- Advantages: Simple queries, excellent query performance
- Disadvantages: Data redundancy, more complex ETL

### 6.1.2 Snowflake Schema

- Normalized dimension tables to 3NF
- Reduced dimension table storage
- More complex joins required for queries
- Use when dimension tables are extremely large

### 6.1.3 Fact Table Design

- Granularity definition: Transaction vs. daily summary facts
- Measure selection: Additive, semi-additive, non-additive
- Foreign key relationships: Links to dimensions
- Degenerate dimensions: Low-cardinality attributes in fact table
- Factless fact tables: Many-to-many relationships

## 6.2 Aggregate and Summary Tables

### 6.2.1 Pre-Aggregated Metrics

Pre-compute common aggregations to improve dashboard and report performance.

- Daily/monthly/quarterly summaries
- Customer-level aggregates: Total spending, purchase count
- Product-level aggregates: Units sold, revenue by category
- Time-series snapshots: Maintain historical snapshots

### 6.2.2 Incremental Load Patterns

- Watermark-based: Load data modified since last run
- Delta computation: Calculate only changes from previous period
- Upsert patterns: Update existing records or insert new
- Materialized view refresh: Incrementally update views

## 6.3 Azure Synapse Analytics for Gold Layer

### 6.3.1 Dedicated SQL Pools

- Purpose-built data warehouse for BI workloads
- Distribution strategies: Round-robin, hash, replicated
- Table design: Heap vs. Clustered Column Store Index (CCI)
- Scaling: Pausable compute resources, on-demand scaling
- Typical use: BI tools directly querying fact/dimension tables

### 6.3.2 Serverless SQL Pools

- Query ADLS data directly without loading into warehouse
- Parquet, CSV, JSON, Delta format support
- Ad-hoc analytics and data exploration
- Cost per query - no reserved capacity needed

### 6.3.3 Loading Patterns in Synapse

- PolyBase: Load from ADLS Gen2 using T-SQL
- Copy command: Simplified syntax for ADLS loading
- Data Factory pipeline integration: Orchestrated loading
- Temp tables: Staging area before final insert

# 7. Advanced Processing - Databricks & Spark

## 7.1 Azure Databricks Integration with ADF

### 7.1.1 Databricks Workspace Setup

Azure Databricks is a unified analytics platform with Apache Spark at its core, integrated with Azure services.

- Create Databricks workspace within Azure subscription
- Configure cluster policies for cost management
- Set up secret scope for secure credential management
- Configure Unity Catalog for data governance
- Integrate with Azure AD for authentication

### 7.1.2 ADF and Databricks Integration

- Databricks Notebook activity: Execute Python or SQL notebooks
- Databricks Spark Job activity: Run JAR files or Python scripts
- Parameter passing: Inject pipeline variables into notebooks
- Error handling: Capture notebook failures and implement retries

## 7.2 PySpark Transformations for Silver Layer

### 7.2.1 DataFrame Operations

- Read from ADLS Gen2: spark.read.parquet() or spark.read.csv()
- Transformation chains: Filter, select, groupBy, join operations
- Window functions: Row number, rank, lag/lead operations
- UDF (User Defined Functions): Custom business logic

### 7.2.2 Data Quality Functions

- removeNull(): Drop rows with null values in key columns
- dropDuplicates(): Remove duplicate rows based on columns
- fillna(): Fill null values with defaults or computed values
- withColumn(): Add new columns with derived values
- Cast operations: Convert data types safely

### 7.2.3 Performance Optimization

- Partitioning: Divide data for parallel processing
- Bucketing: Organize data for join optimization
- Caching: Cache frequently accessed dataframes
- Broadcast joins: For small dimension tables
- Shuffle optimization: Minimize data movement across partitions

## 7.3 Structured Streaming for Real-Time Processing

### 7.3.1 Streaming Data Ingestion

Process continuous data streams from Event Hubs or Kafka in real-time.

- readStream source: Configure Event Hubs or Kafka connection
- Windowed operations: Aggregate over time windows
- Watermarking: Handle late-arriving data gracefully
- Output modes: Append, update, or complete

### 7.3.2 Stream Processing with Databricks

- Micro-batch processing: Process data in small batches
- Continuous mode: Row-by-row processing (experimental)
- Exactly-once semantics: Ensure no data loss or duplication
- Join streams: Combine multiple streaming sources

# 8. Analytics & Business Intelligence Layer

## 8.1 Power BI Integration with Azure Data Engineering

### 8.1.1 Power BI Semantic Models

Power BI semantic models (formerly called datasets) define the structure for analytics. They connect to Gold layer data and establish relationships.

- Fact tables: Import measures from Gold layer fact tables
- Dimension tables: Import attributes from dimension tables
- Relationships: Define cardinality and join properties
- Hierarchies: Create drill-down navigation paths
- DAX measures: Calculated metrics and KPIs

### 8.1.2 Direct Query vs. Import

- Import mode: Copy data into Power BI for fast refresh, limited size
- DirectQuery mode: Query Synapse pools directly, always current
- Aggregation tables: Combine approaches for performance

- Hybrid approach: Import small dimensions, DirectQuery fact tables

### 8.1.3 Power BI Dataflows

- Self-service ETL: Business users create transformations
- Reusable dataflows: Share across multiple Power BI reports
- Dataflow storage: ADLS integration for shared storage
- Linked entities: Build on other dataflows

## 8.2 Dashboard and Report Development

### 8.2.1 Semantic Model Best Practices

- Hide unnecessary columns and measures from end users
- Implement Row-Level Security (RLS) for data governance
- Create role-based filtering for sensitive data
- Optimize model performance: Reduce cardinality, use summarize

### 8.2.2 Report Design Patterns

- Executive summary: One-page overview of key metrics
- Detailed analytics: Multi-page drill-down reports
- Operational dashboards: Real-time KPI monitoring
- Ad-hoc analysis: Unpinned pages for exploration
- Mobile layout: Optimized for mobile device consumption

### 8.2.3 Refresh Scheduling

- Full refresh: Reload all data (typically daily)
- Incremental refresh: Update only changed data (hourly/more frequently)
- Push datasets: Real-time updates for streaming scenarios
- Dataflow refresh scheduling in app service

# 9. Monitoring, Logging & Data Governance

## 9.1 Azure Monitor Integration

### 9.1.1 ADF Pipeline Monitoring

Comprehensive monitoring of ADF pipelines ensures visibility into data engineering operations.

- Pipeline runs: Track start time, duration, status
- Activity performance: Individual task execution details
- Data profiling: Row counts, column statistics
- Error tracking: Failed activities with error messages
- Integration Runtime monitoring: Compute resource utilization

### 9.1.2 Log Analytics Integration

- Send diagnostic logs to Log Analytics workspace
- Create KQL (Kusto Query Language) queries for analysis
- Build dashboards for operational insights
- Set up alerts based on log analysis
- Long-term log retention for compliance

### 9.1.3 Application Insights

- Track Data Flow execution metrics
- Monitor stored procedure execution in Synapse
- Capture custom metrics from Azure Functions
- Distributed tracing across services

## 9.2 Data Quality Monitoring & Observability

### 9.2.1 Data Quality Rules

Define and monitor business rules to ensure ongoing data quality.

- Row count validation: Compare source to target record counts
- Null percentage thresholds: Alert if nulls exceed tolerance
- Value distribution: Monitor for unexpected patterns
- Referential integrity: Check foreign key relationships
- Business rule validation: Domain-specific data checks

### 9.2.2 Data Quality Dashboard

- Track quality score over time
- Visualize failed rule execution
- Show data freshness metrics
- Display pipeline execution timelines
- Anomaly detection highlighting

## 9.3 Azure Purview for Data Governance

### 9.3.1 Data Catalog

Azure Purview provides enterprise data governance with discovery, classification, and lineage.

- Automated scanning: Discover data assets across Azure
- Classification: Apply system and custom classification labels
- Glossary: Define business terms and terminology
- Sensitivity labels: PII, confidential, public classifications

### 9.3.2 Lineage and Impact Analysis

- Track data flow from source to destination
- Understand dependencies between datasets
- Impact analysis: Determine downstream effects of changes
- Audit trails: Track who accesses which data

### 9.3.3 Data Policy Enforcement

- Set access policies on sensitive data assets
- Role-based access control (RBAC) integration
- Data use agreements: Document approved use cases
- Compliance mapping: Align with regulations (GDPR, HIPAA)

# 10. Security & Compliance Implementation

## 10.1 Network Security

### 10.1.1 Azure Virtual Network Integration

Secure Azure data engineering components through network isolation.

- Virtual Networks (VNets): Isolate Azure services from internet
- Subnets: Segment network by function and security zone
- Network Security Groups (NSGs): Firewall rules for traffic control
- Private Endpoints: Private connectivity to Azure services
- Service Endpoints: Restrict data access to specific VNets

### 10.1.2 Managed Virtual Networks (ADF/Databricks)

- Create isolated network environment for ADF
- Automatically manage private endpoints for linked services
- Eliminate outbound connectivity to internet
- Improve security posture for regulated environments

## 10.2 Identity and Access Management

### 10.2.1 Managed Identities

Use system-managed or user-managed identities for secure service authentication.

- System-managed identity: Azure automatically manages credentials
- User-managed identity: Long-lived credential for multiple services
- No password management required
- Automatic token refresh

### 10.2.2 Azure AD Integration

- Authenticate to Azure services using Azure AD credentials
- Conditional access policies: Require MFA or device compliance
- Privileged Identity Management (PIM): Just-in-time access
- Service principals: Application identity for automation

### 10.2.3 Role-Based Access Control (RBAC)

- Storage Blob Data Owner: Full permissions on containers
- Storage Blob Data Contributor: Read and write access
- Storage Blob Data Reader: Read-only access
- Custom roles: Define specific permission combinations

## 10.3 Data Encryption

### 10.3.1 Encryption at Rest

- Azure Storage: Automatic encryption with Microsoft-managed keys
- Customer-managed keys: Use Azure Key Vault for key control
- Transparent Data Encryption (TDE): Database encryption
- Always Encrypted: Column-level encryption for sensitive data

### 10.3.2 Encryption in Transit

- TLS 1.2+: All data movement encrypted
- HTTPS: Web service communications
- SSL/TLS certificates: Secure data transfers
- VPN/ExpressRoute: Encrypted on-premises to Azure connectivity

## 10.4 Compliance and Audit

### 10.4.1 Audit Logging

- Azure Storage logging: Track access to data lake files
- ADF audit events: Pipeline execution and modifications
- SQL audit: Database access and modifications
- Azure AD sign-in logs: User authentication events

### 10.4.2 Compliance Frameworks

- GDPR: Right to be forgotten, data portability
- HIPAA: Healthcare data protection requirements
- PCI-DSS: Payment card industry data security
- SOC 2: Security and availability controls

# 11. Testing & Validation Framework

## 11.1 Data Quality Validation

### 11.1.1 Schema Validation

Validate that data structure matches expectations at each layer.

- Column existence: Verify all expected columns present
- Data types: Confirm columns have correct types
- Nullable constraints: Check if nulls allowed
- Length validation: String columns meet length requirements

### 11.1.2 Business Rule Validation

- Referential integrity: Foreign key relationships valid
- Value range checks: Data within acceptable ranges
- Uniqueness: Key columns contain no duplicates
- Cardinality: Expected number of distinct values
- Temporal logic: Dates in correct sequences

### 11.1.3 Statistical Validation

- Null percentage: Monitor percentage of null values
- Outlier detection: Identify statistical anomalies
- Distribution analysis: Check for unexpected patterns
- Trend analysis: Identify sudden changes in metrics

## 11.2 ETL Testing Patterns

### 11.2.1 Unit Testing

- Test individual transformations in isolation
- Data Flow expression testing: Test custom expressions
- PySpark notebook testing: Unit test Spark logic
- SQL stored procedure testing: Test T-SQL functions

### 11.2.2 Integration Testing

Test complete data flows from source to target.

- End-to-end pipeline testing: Full medallion flow
- Data reconciliation: Source vs. target counts
- Transformation accuracy: Verify calculations correct
- Performance testing: Measure execution time

### 11.2.3 Regression Testing

- Test after changes to ensure no breaking issues
- Historical data validation: Reprocess previous periods
- Snapshot comparison: Compare current with expected output
- Automated test suites: CI/CD pipeline integration

## 11.3 User Acceptance Testing (UAT)

### 11.3.1 UAT Planning

- Identify business stakeholders and subject matter experts
- Define test cases based on business requirements
- Create test data sets representing real scenarios
- Document expected outcomes and acceptance criteria

### 11.3.2 UAT Execution

- Provide isolated test environment (non-production)
- Train users on data access and tools
- Execute test cases and document results
- Address discrepancies between expected and actual

# 12. Deployment & CI/CD Pipeline

## 12.1 Infrastructure as Code (IaC)

### 12.1.1 Azure Resource Manager (ARM) Templates

Define Azure infrastructure in JSON templates for reproducible deployments.

- Storage accounts: ADLS Gen2 configuration
- Data Factory: Pipeline, linked services, datasets
- Synapse workspaces: SQL pools and compute resources
- Networking: VNets, private endpoints, firewalls

### 12.1.2 Terraform Configuration

- Declare infrastructure in HCL (HashiCorp Configuration Language)
- Infrastructure versioning in Git repositories
- State management: Track infrastructure state
- Modularity: Reusable infrastructure components

### 12.1.3 Environment Parity

- Dev environment: Fast feedback, frequent changes
- Staging environment: Production-like testing
- Production environment: Live data, security lockdown
- Parameterization: Environment-specific variables

## 12.2 CI/CD Pipeline with Azure DevOps

### 12.2.1 Version Control

Maintain complete version history of data engineering artifacts.

- Git repositories: ADF pipelines, Databricks notebooks, SQL scripts
- Branching strategy: Feature branches, main branch protection
- Pull requests: Code review before merging
- Commit history: Audit trail of changes

### 12.2.2 Build Pipeline

- Validate ARM templates: Syntax and structure validation
- Run unit tests: Test transformations locally
- Code quality: Static analysis and linting
- Security scanning: Detect hardcoded secrets or vulnerabilities

### 12.2.3 Release Pipeline

- Deploy to dev environment: Automated from feature branch
- Deploy to staging: Integration testing environment
- Manual approval: Gate for production deployment
- Deploy to production: Automated deployment to live

## 12.3 GitHub Actions for CI/CD

### 12.3.1 Workflow Configuration

- Trigger on pull requests: Automated testing before merge
- Trigger on main branch push: Deployment automation
- Scheduled workflows: Regular validation runs
- Manual trigger: On-demand deployments

### 12.3.2 Azure DevOps Integration

- Service connection: Authenticate to Azure subscription
- Deployment groups: Target deployment environments
- Artifact repository: Store build outputs
- Deployment slots: Blue-green deployments

# 13. Production Deployment & Go-Live

## 13.1 Pre-Deployment Checklist

### 13.1.1 Technical Validation

Comprehensive technical verification before production deployment.

- Data validation: Reconciliation between environments
- Performance testing: Meets SLA requirements
- Security validation: Encryption, access controls, compliance
- Disaster recovery: Backup and recovery tested
- Monitoring setup: Alerts and dashboards configured

### 13.1.2 Documentation Review

- Architecture documentation: Current and accurate
- Runbooks: Operational procedures documented
- Data dictionary: Business glossary complete
- Change log: Version history and updates

### 13.1.3 Stakeholder Sign-Off

- Data governance: Approval from data stewards
- Business stakeholders: Confirmation of data accuracy
- IT operations: Readiness to support in production
- Security and compliance: Authorization to deploy

## 13.2 Cutover Strategy

### 13.2.1 Big Bang Approach

- Advantages: Fast cutover, single truth source immediately
- Disadvantages: High risk if issues discovered
- Requirements: Extensive testing and validation
- Rollback: Complete data reload and revert processes

### 13.2.2 Phased Cutover

- Wave 1: Limited data or users
- Wave 2: Expand to larger subset
- Wave 3: Full production load
- Advantages: Risk reduction, identified issues addressed

### 13.2.3 Parallel Run

- Run new pipeline alongside existing system
- Compare outputs for accuracy verification
- Maintain old system until confidence achieved
- Requires additional infrastructure and effort

## 13.3 Production Handover

### 13.3.1 Operations Team Training

- Hands-on training on monitoring and alerting

- Troubleshooting procedures for common issues
- Escalation procedures for critical failures
- Knowledge transfer documentation

#### 13.3.2 Support Model

- On-call schedule: 24/7 support for critical issues
- SLA definition: Response and resolution times
- Incident management: Documented procedures
- Change control: Approval process for modifications

#### 13.3.3 Monitoring & Alerting Handoff

- Dashboard access: Provide operations team access
- Alert routing: Configure for operations team
- Dashboard training: Interpretation of metrics
- Contact list: Technical contacts for escalation

# 14. Post-Implementation & Optimization

## 14.1 Performance Tuning

### 14.1.1 Pipeline Performance Analysis

Optimize data engineering pipelines for cost and speed.

- Execution time profiling: Identify slow activities
- Data volume analysis: How much data moved/processed
- Parallelism optimization: Number of cores and workers
- Memory optimization: Reduce memory footprint

### 14.1.2 Query Performance Optimization

- Index analysis: Create indexes on join and filter columns
- Statistics: Update table statistics for query optimizer
- Execution plan analysis: Identify inefficient operations
- Data distribution: Ensure even distribution for MPP systems

### 14.1.3 Spark Optimization

- Shuffle optimization: Reduce data movement across partitions
- Broadcast joins: Use for small dimension tables
- Partition pruning: Filter partitions during reads
- Memory tuning: Driver and executor memory configuration

## 14.2 Cost Optimization

### 14.2.1 Azure Storage Cost Reduction

Optimize data lake storage costs through smart data management.

- Tiered storage: Move cold data to Archive tier
- Compression: Reduce data volume using Parquet format
- Data lifecycle policies: Automatic tier transitions
- Deduplication: Remove redundant data

### 14.2.2 Compute Cost Optimization

- ADF integration runtime sizing: Right-size compute resources
- Databricks cluster optimization: Use spot instances for non-critical jobs
- Synapse pause: Pause dedicated pools during non-business hours
- Scheduled scaling: Resize pools based on workload patterns

### 14.2.3 Data Transfer Cost Reduction

- Keep data in region: Minimize egress bandwidth charges
- Data compression: Reduce network transfer size
- Connection optimization: Use private endpoints where possible
- Batch transfers: Combine small transfers into batches

## 14.3 Continuous Improvement Process

### 14.3.1 Monitoring Metrics Review

- Weekly performance reviews: Pipeline execution time trends
- Cost analysis: Identify high-cost operations
- Data quality metrics: Track quality improvement
- User feedback: Collect analytics user requests

### 14.3.2 Capacity Planning

- Forecast data growth: Project future data volumes
- Scalability assessment: Plan infrastructure expansion
- Workload distribution: Balance analytical and transactional
- Technology evaluation: Assess newer Azure services

### 14.3.3 Feature Enhancements

- New data source integration: Add new business domains
- Granularity enhancement: Increase data detail level
- Real-time analytics: Add streaming capabilities
- Advanced analytics: Implement ML models

# 15. Disaster Recovery & Business Continuity

## 15.1 Backup Strategy

### 15.1.1 Data Backup

Implement comprehensive backup strategy for data recovery.

- ADLS Gen2 backups: Point-in-time restore capability
- Synapse backups: Automatic daily backups and restore points
- Database backups: Full, differential, and transaction log backups
- Backup frequency: Align with RPO (Recovery Point Objective)

### 15.1.2 Backup Storage

- Geo-redundant storage (GRS): Copies to secondary region
- Read-access geo-redundancy (RA-GRS): Read-only secondary
- Separate location: Off-site backup vault

- Long-term retention: Annual backups for compliance

### 15.1.3 Backup Testing

- Regular restore testing: Validate backup integrity
- Point-in-time recovery testing: Restore specific times
- Document recovery procedures: Step-by-step guides
- Recovery timing validation: Measure actual recovery time

## 15.2 Disaster Recovery Planning

### 15.2.1 RTO and RPO Objectives

Define recovery objectives based on business criticality.

- RTO (Recovery Time Objective): Maximum acceptable downtime
- RPO (Recovery Point Objective): Maximum acceptable data loss
- Critical systems: Low RTO/RPO (hours/minutes)
- Non-critical systems: Higher RTO/RPO (days)

### 15.2.2 Disaster Recovery Architecture

- Cold standby: Backup infrastructure on-demand (high RTO)
- Warm standby: Partial infrastructure always running
- Hot standby: Full redundancy with real-time failover
- Geo-redundancy: Failover to secondary region

### 15.2.3 Failover Procedures

- Automatic failover: Detect failures and switch automatically
- Manual failover: Operator-initiated after approval
- DNS update: Point applications to backup infrastructure
- Validation: Verify backup systems operational

## 15.3 Business Continuity Plan

### 15.3.1 Continuity Measures

- Load balancing: Distribute across availability zones
- Redundancy: No single point of failure
- Health checks: Continuous monitoring for failures
- Circuit breakers: Stop cascading failures

### 15.3.2 Testing and Drills

- Annual DR drill: Full failover and recovery exercise
- Tabletop exercises: Team discusses scenarios
- Document findings: Improve procedures based on results
- Keep plan current: Update for infrastructure changes

# 16. Operational Excellence & Best Practices

## 16.1 Naming Conventions & Standards

### 16.1.1 Storage & Container Naming

- Storage account: {environment}{purpose}{uniqueid} (e.g., proddelakemain01)
- Containers: Lowercase, hyphens for spaces (e.g., bronze-sales-data)
- Folder paths: Layer/domain/source/table/YYYY/MM/DD
- Files: Descriptive names with timestamps (e.g., sales_20240528_1430.parquet)

### 16.1.2 Data Factory Naming

- Pipelines: {source}_{operation}_{frequency} (e.g., AdventureWorks_BronzeIngest_Daily)
- Datasets: {layer}_{source}_{entity} (e.g., Bronze_AdventureWorks_Sales)
- Linked Services: {service_type}_{environment} (e.g., AzureSqlDatabase_Prod)
- Activities: Clear, descriptive names for each activity

### 16.1.3 Database Object Naming

- Schemas: Organize by domain (e.g., sales, marketing, finance)
- Tables: Descriptive singular names (e.g., fact_sales, dim_customer)
- Columns: Lowercase with underscores (e.g., customer_id, created_date)
- Procedures: {layer}_{operation} (e.g., silver_clean_sales)

## 16.2 Documentation & Knowledge Management

### 16.2.1 Architecture Documentation

- Solution architecture: System diagram with data flows
- Data lineage: Source to destination for all datasets
- Medallion layer mapping: Bronze to Silver to Gold transformations
- Integration patterns: How systems connect and communicate

### 16.2.2 Operational Runbooks

- Daily operations: Standard operating procedures
- Troubleshooting: Common issues and resolutions
- Escalation paths: Who to contact for different issues
- Emergency procedures: Steps for critical failures

### 16.2.3 Data Dictionary & Glossary

- Business terms: Standardized terminology
- Data definitions: What each field contains
- Calculation formulas: How metrics are derived
- Data quality rules: Validation and business rules

## 16.3 Cost Management

### 16.3.1 Cost Monitoring

Track and optimize cloud spending for data engineering services.

- Cost allocation tags: Tag resources by cost center
- Budget alerts: Configure spending thresholds

- Usage analytics: Analyze consumption patterns
- Reserved instances: Commit to long-term savings

### 16.3.2 Right-Sizing Resources

- Analyze actual utilization vs. provisioned capacity
- Adjust cluster sizes based on workload demands
- Use auto-scaling where available
- Periodic review: Quarterly cost optimization reviews

## 16.4 Team Structure & Roles

### 16.4.1 Data Engineering Team

- Principal Data Engineer: Architecture and strategy
- Senior Data Engineer: Complex transformation design
- Data Engineer: Pipeline development and maintenance
- Data Engineering Support: Infrastructure and operations

### 16.4.2 Collaboration & Communication

- Daily standup: Brief sync on progress and blockers
- Technical design review: Peer review of architectures
- Incident post-mortems: Learning from failures
- Quarterly planning: Roadmap and capacity planning

# 17. Azure Data Engineering Service Reference Guide

## 17.1 Data Ingestion Services

| Service | Purpose |
|---|---|
| Azure Data Factory | Orchestrate and automate ETL/ELT pipelines |
| Event Hubs | Real-time event streaming and ingestion |
| Stream Analytics | Real-time stream processing and analytics |
| IoT Hub | IoT device data ingestion at scale |

## 17.2 Data Storage Services

| Service | Purpose |
|---|---|
| ADLS Gen2 | Scalable data lake for Bronze/Silver layers |
| Synapse Analytics | Integrated analytics with SQL and Spark |
| Azure SQL Database | Relational database for operational/Gold layer |
| Cosmos DB | NoSQL database for semi-structured data |

## 17.3 Processing & Analytics Services

| Service | Purpose |
|---|---|
| Databricks | Apache Spark for distributed processing |
| Synapse Spark Pools | Spark processing within Synapse workspace |
| Azure Functions | Serverless compute for lightweight operations |

| | |
|---|---|
| Machine Learning | ML model development and deployment |

## 17.4 Governance & Monitoring Services

| Service | Purpose |
|---|---|
| Azure Purview | Data governance, cataloging, and lineage |
| Azure Monitor | Unified monitoring and alerting |
| Log Analytics | Log aggregation and KQL querying |
| Application Insights | Application performance monitoring |

*End of Document*